161.777 Practical Data Mining

Assignment 1, 2026

```
# Add any other packages you need to load here.
library(tidyverse)

# Read in the data
climate <- read_csv("https://www.massey.ac.nz/~jcmarsha/161777/data/climate.csv")
athletes <- read_csv("https://www.massey.ac.nz/~jcmarsha/161777/data/athletes.csv")
```

THIS FILE IS WHERE YOU WRITE YOUR ANSWERS. THE QUESTIONS ARE AVAILABLE ON STREAM

Exercise 1: Analysis of Incomplete Temperature

1.1. Precise year and month

```
# a. The last observation was made at location A
climate |>
  select(Year, Month,A) |> # Select the data for location A.
  arrange(desc(Year),desc(Month)) |> # Arrange the data so that the last observation appears first.
  filter(!is.na(A)) |>  # Remove rows with missing values (NA), keeping only valid observations.
  mutate(P_year=Year+1863-1,P_month=Month) |> # Create a new column for the exact year and add a month column.Keep  the month n
umbers unchanged; only rename the column to align with the year column.
  select(P_year,P_month) |> # Keep only the calculated year and month columns.
  slice_max(P_year,n=1) |> # Select the most recent year (last observation).
  slice_max(P_month,n=1) # Select the most recent month (last observation).
```

```
## # A tibble: 1 × 2
##   P_year P_month
##    <dbl>   <dbl>
## 1   1924       3
```

```
# a. The last observation was made at location A (make_datetime function)
library(lubridate)
climate|>
  select(Year,Month,A) |>  # Select the data for location A.
  filter(!is.na(A)) |>   # Remove rows with missing values (NA), keeping only valid observations.
  mutate(time=make_datetime(year=1863+Year-1,month=Month,day = 1)) |>  # Create a new column representing the precise year(new
function).
  slice_max(time,n=1) |>   # choose the largest time, get the time directly or continue next step.
  mutate(P_year=year(time),P_month=month(time)) |>  # Create two columns-year and month.
  select(P_year,P_month)  # Select the last observation for location A.
```

```
## # A tibble: 1 × 2
##   P_year P_month
##    <dbl>   <dbl>
## 1   1924       3
```

```
# b. The first observation was made at location B
climate |>
select(Year, Month,B)|>  # Choose B data set.
  filter(!is.na(B)) |>  # Delete the data is NA.
  arrange(Year,Month) |>   # Identify the earliest year and month.
  mutate(F_year=min(Year),F_month=min(Month)) |> # Create columns for the earliest year and month.
  select(F_year,F_month) |>   # Retain only the relevant columns.
  slice_sample(n=1) |>   # Select the first row.
  mutate(P_year=F_year+1863-1,P_month=F_month) |>  # Compute the precise year and month.
  select(P_year,P_month)    # Select the first observation for location B.
```

```
## # A tibble: 1 × 2
##   P_year P_month
##    <dbl>   <dbl>
## 1   1924       1
```

```
# Location A ends in March 1924, while location B begins in January 1924, which matches the description "only three months over
the entire period for which data are available at both A and B".
```

```
# b. The first observation was made at location B (make_datetime function)
library(lubridate)
climate|>
  select(Year,Month,B) |>   # Choose B data set.
  filter(!is.na(B)) |>   # Delete the data is NA.
  mutate(time=make_datetime(year=Year+1863-1,month=Month)) |>    # Compute the precise year and month.
  arrange(time) |>  # Identify the earliest year and month.
  select(time) |>   # Retain only the relevant columns.
  slice_min(time,n=1) |>    # Select the first row,get the time directly or continue next step.
  mutate(P_year=year(time),P_month=month(time)) |> # Create two columns-year and month.
  select(P_year, P_month) # Select the first observation for location B.
```

```
## # A tibble: 1 × 2
##   P_year P_month
##    <dbl>   <dbl>
## 1   1924       1
```

1.2. Table of mean temperature by month for locations C and D

```
# Mean temperature by month in locations C and D - month.abb.
climate |>
  select(Month, C, D) |>  #Select the data set
  filter(!is.na(C)) |>    #Ensure that the C column contains numeric values.
  filter(!is.na(D)) |>    #Ensure that the D column contains numeric values.
  group_by(Month)|>    #Group the data by month.
  summarise(mean_c=mean(C),mean_d=mean(D)) |> #Calculate the mean temperatures for C and D by month.
  mutate(month=month.abb[Month]) |> #Add a column with descriptive month names using a month vector.
  select(month,mean_c,mean_d) #Retain only the required information.
```

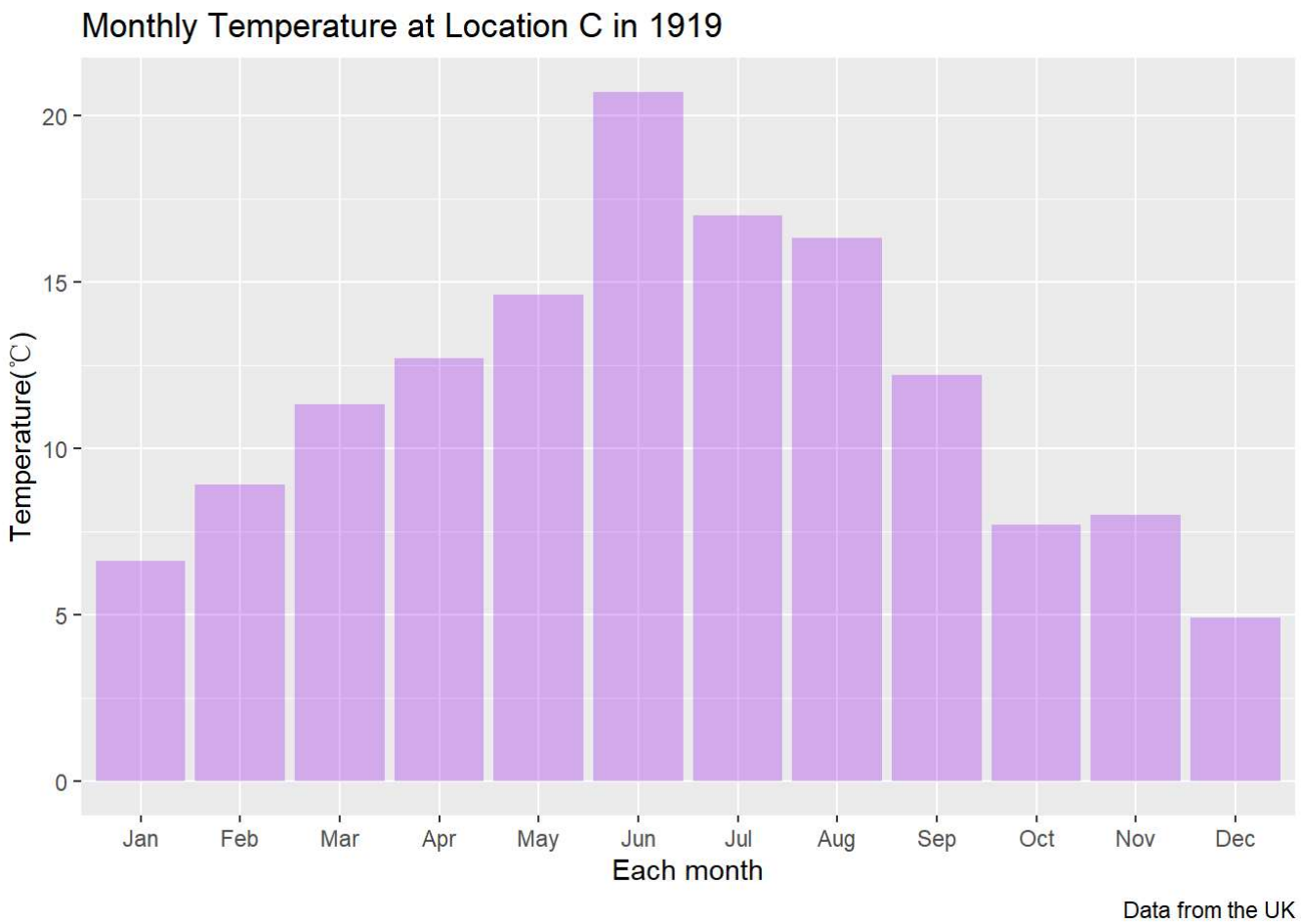
```
## # A tibble: 12 × 3
##    month mean_c mean_d
##    <chr>  <dbl>  <dbl>
##  1 Jan     7.01   3.59
##  2 Feb     9.10   3.77
##  3 Mar    10.8    5.98
##  4 Apr    13.8    7.86
##  5 May    15.8   11.0
##  6 Jun    17.9   13.4
##  7 Jul    18.0   15.4
##  8 Aug    15.2   15.5
##  9 Sep    12.3   12.8
## 10 Oct     8.99   9.39
## 11 Nov     7.40   6.00
## 12 Dec     6.86   4.29
```

```
# Mean temperature by month in locations C and D - month.name
climate |>
  select(Month, C, D) |>  # Select the data set
  group_by(Month)|>    # Group the data by month.
  summarise(mean_c=mean(C),mean_d=mean(D),na.rm=TRUE) |> # Calculate the mean temperatures for C and D by month,  # Ensure that the C D column contains numeric values.
  mutate(month=month.name[Month] )|># Add a column with descriptive month names.
  select(month,mean_c,mean_d) # Retain only the required information.
```

```
## # A tibble: 12 × 3
##    month     mean_c mean_d
##    <chr>      <dbl>  <dbl>
##  1 January     7.01   3.59
##  2 February    9.10   3.77
##  3 March      10.8    5.98
##  4 April      13.8    7.86
##  5 May        15.8   11.0
##  6 June       17.9   13.4
##  7 July       18.0   15.4
##  8 August     15.2   15.5
##  9 September  12.3   12.8
## 10 October     8.99   9.39
## 11 November    7.40   6.00
## 12 December    6.86   4.29
```

1.3. Monthly temperatures at location C in 1919

```
# Produce a bar plot of the monthly temperatures at location C in 1919.
climate |>
  mutate(year=1863+Year-1) |>  # Compute the precise year and month.
  filter(year==1919) |>  # Choose the aim year.
  mutate(month=month.abb[Month])|>  # Numeric values (1 - 12) are not intuitive for representing months.Therefore, convert them
to month names.
  select(month,C)|>   # Keep only the necessary data for plotting.
  rename(temperature_c=C) |>  # Using “C” directly as an axis label is unclear, so it is renamed for better interpretation.
# Ensure months are ordered correctly (Jan to Dec) for plotting.
  mutate(month = fct_relevel(month, "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                          "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) |>
# Use the fct_relevel() function to define the month order, because when I finished the code, the months were messy, so use the
last visualization practice function from Workshop 2.
  ggplot(mapping=aes(x=month,y=temperature_c))+  # Define x,y data.
  geom_col(fill='Purple',alpha=0.3)+  # Change the color more clear to show in a low transparency.
  labs(x="Each month",
       y="Temperature(℃)",
       title="Monthly Temperature at Location C in 1919",
       caption="Data from the UK") #Add axis labels,title,caption with labs().
```



#From this plot, it is clear that the temperature in 1919 ranged between 5° C and 20° C throughout the year. The highest temperature occurred in June, while the lowest was in December. The weather was warmer in summer, whereas it was colder in winter and early spring.

1.4. Monthly temperatures in 1924 for all locations

```
#Step 1: Impute missing values using k-Nearest Neighbors (k = 5).
library(VIM)   # Load necessary library.
```

```
## Loading required package: colorspace
```

```
## Loading required package: grid
```

```
## VIM is ready to use.
```

```
## Suggestions and bug-reports can be submitted at: https://github.com/statistikat/VIM/issues
```

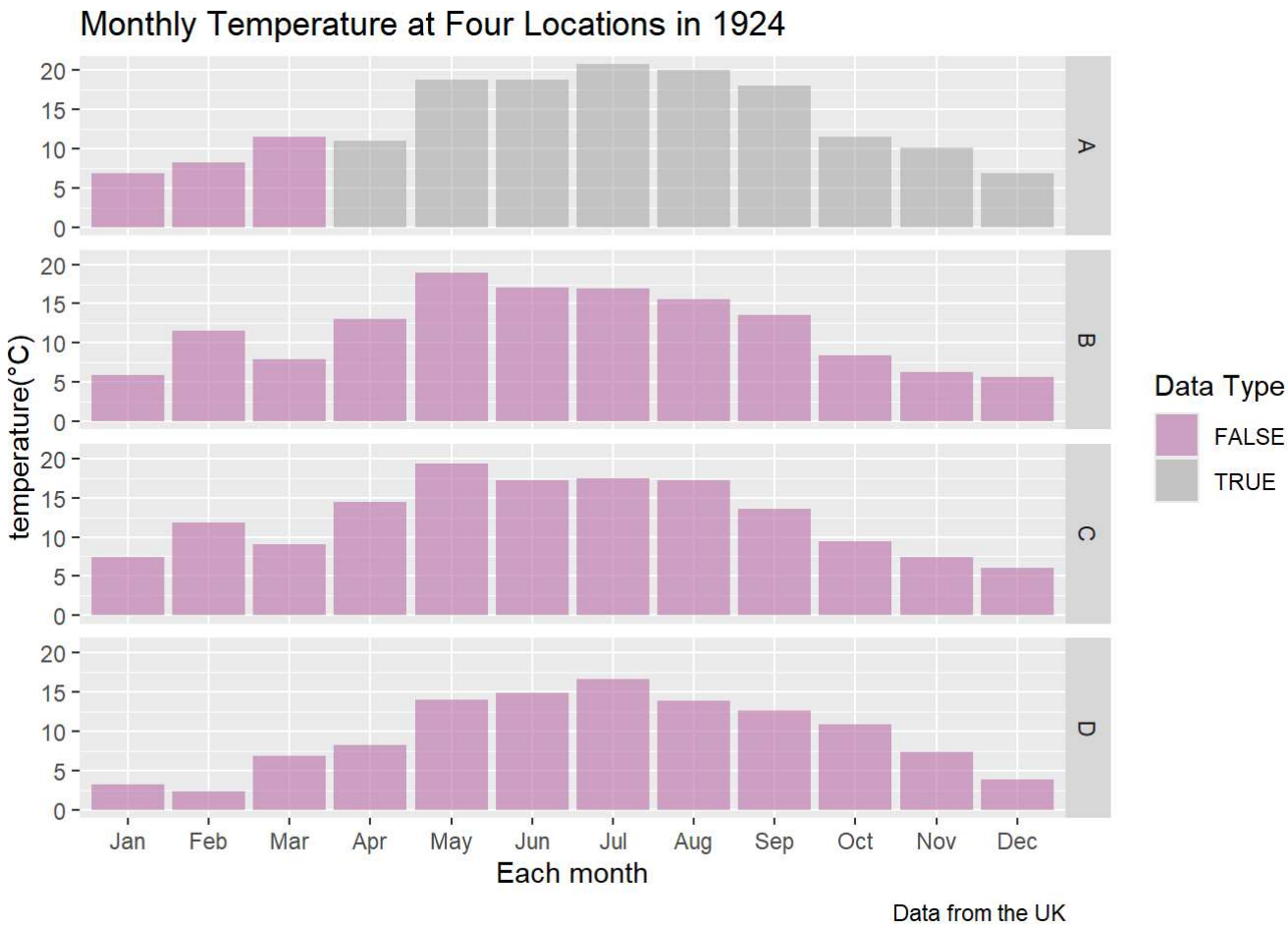
```
##
## Attaching package: 'VIM'
```

```
## The following object is masked from 'package:datasets':
##
##     sleep
```

```
climate_na <- climate |>
  kNN(k=5)  # Replace missing values based on similarity with 5 nearest observations.

# Step 2: Select and prepare data for the target year (1924).
climate_1924 <- climate_na |>
  mutate(year = 1863 + Year - 1) |>  # Convert the original Year index into actual calendar year.
  filter(year == 1924) |>   # Filter data for the year 1924.
  mutate(month = month.abb[Month]) |>   # Convert numeric month (1-12) into month abbreviations (Jan-Dec).
# Ensure months are ordered correctly (Jan to Dec) for plotting.
  mutate(month = fct_relevel(month, "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) |>
  select(month, A, B, C, D, A_imp, B_imp, C_imp, D_imp) # Keep only relevant variables.
climate_1924
```

```
##    month    A    B    C    D A_imp B_imp C_imp D_imp
## 1    Jan  6.9  5.9  7.4  3.2 FALSE FALSE FALSE FALSE
## 2    Feb  8.3 11.6 11.8  2.3 FALSE FALSE FALSE FALSE
## 3    Mar 11.5  7.9  9.1  6.8 FALSE FALSE FALSE FALSE
## 4    Apr 11.0 13.0 14.5  8.2  TRUE FALSE FALSE FALSE
## 5    May 18.8 18.9 19.4 14.0  TRUE FALSE FALSE FALSE
## 6    Jun 18.8 17.1 17.2 14.8  TRUE FALSE FALSE FALSE
## 7    Jul 20.8 16.9 17.5 16.6  TRUE FALSE FALSE FALSE
## 8    Aug 20.1 15.6 17.2 13.9  TRUE FALSE FALSE FALSE
## 9    Sep 18.0 13.5 13.6 12.6  TRUE FALSE FALSE FALSE
## 10   Oct 11.5  8.4  9.5 10.8  TRUE FALSE FALSE FALSE
## 11   Nov 10.2  6.3  7.4  7.3  TRUE FALSE FALSE FALSE
## 12   Dec  6.9  5.7  6.1  3.8  TRUE FALSE FALSE FALSE
```

```
# Produce a bar plot of the monthly temperatures at all four locations in 1924.

#Step 1: Impute missing values using k-Nearest Neighbors (k = 5).
library(VIM)   # Load necessary library.
climate_na <- climate |>
  kNN(k=5)  # Replace missing values based on similarity with 5 nearest observations.

# Step 2: Select and prepare data for the target year (1924).
climate_1924 <- climate_na |>
  mutate(year = 1863 + Year - 1) |>  # Convert the original Year index into actual calendar year.
  filter(year == 1924) |>   # Filter data for the year 1924.
  mutate(month = month.abb[Month]) |>   # Convert numeric month (1 - 12) into month abbreviations (Jan - Dec).
# Ensure months are ordered correctly (Jan to Dec) for plotting.
  mutate(month = fct_relevel(month, "Jan", "Feb", "Mar", "Apr", "May", "Jun",
                                    "Jul", "Aug", "Sep", "Oct", "Nov", "Dec")) |>
  select(month, A, B, C, D, A_imp, B_imp, C_imp, D_imp) # Keep only relevant variables.

# Step 3: Create the bar plot dataset (monthly data for each location and data status).
# Convert data from wide to long format (temperature).
climate_values <- climate_1924 |>   # Reshape to obtain temperature values for each location and month temperature values.
  select(month, A, B, C, D) |>      # Keep only temperature-related variables.
  pivot_longer(cols = c(A,B,C,D),       # Columns A - D are gathered into a new "location" column.
               names_to = "location",
               values_to = "temp")

# Convert data from wide to long format(data condition).
climate_imp <- climate_1924 |>    # Reshape to obtain the data status (True/False) for each location and month data condition-r
eal or imputed data.
  select(month, A_imp, B_imp, C_imp, D_imp) |>  # Keep only relevant variables.
  rename(A = A_imp, B = B_imp, C = C_imp, D = D_imp) |>  # Rename columns to match temperature data for merging.
  pivot_longer(
    cols = c(A, B, C, D),     # Gather columns A - D into a "location" column.
    names_to = "location",
    values_to = "is_imputed"    # This logical column defines the data condition (True or False).
  ) |>
  select(month, location, is_imputed)   # Keep only relevant variables for joining.

#Combine temperature values and imputation status.
climate_long <- climate_values |> # Create the final dataset for plotting.
  left_join(climate_imp, by = c("month", "location")) # Join the imputation flag to the temperature dataset by month and locati
on.

# Step 4: Generate the target plot using the prepared dataset.
climate_long|>
  ggplot(mapping=aes(x=month,y=temp,fill=is_imputed))+  # Use 'fill' to distinguish between real and imputed data.
  geom_col(alpha=0.7)+  # Use a bar chart with semi-transparent colors.
  facet_grid(rows=vars(location))+   # Create separate panels for each location.
  scale_fill_manual(values=c("FALSE"="#C582B2","TRUE"="#B7B7B2"))+
# Assign specific colors to real and imputed data (based on Workshop 2).
  labs(x="Each month",     # Set appropriate axis labels and titles.
       y="temperature(° C)",
       fill="Data Type",
       title="Monthly Temperature at Four Locations in 1924",
       caption="Data from the UK")
```




```
# The plot indicates that only Location A contains imputed data from April to December, while all other locations.
# Consist of observed data. Temperatures from May to September are notably higher than in other months.
# The imputed values for Location A appear logically consistent, as they follow a trend similar to the measured data.
```

Exercise 2: Detective Work for Athletes Data

2.1. Find the incorrect entry and obtain the correct value

```
# Question: One entry in the dataset is incorrectly recorded.Identify the data item and suggest the correct value.
# Step 1: Use summary() to check the data.
summary(athletes) #All the data looks logical, with no obvious issues, so we continue to the next check.
```

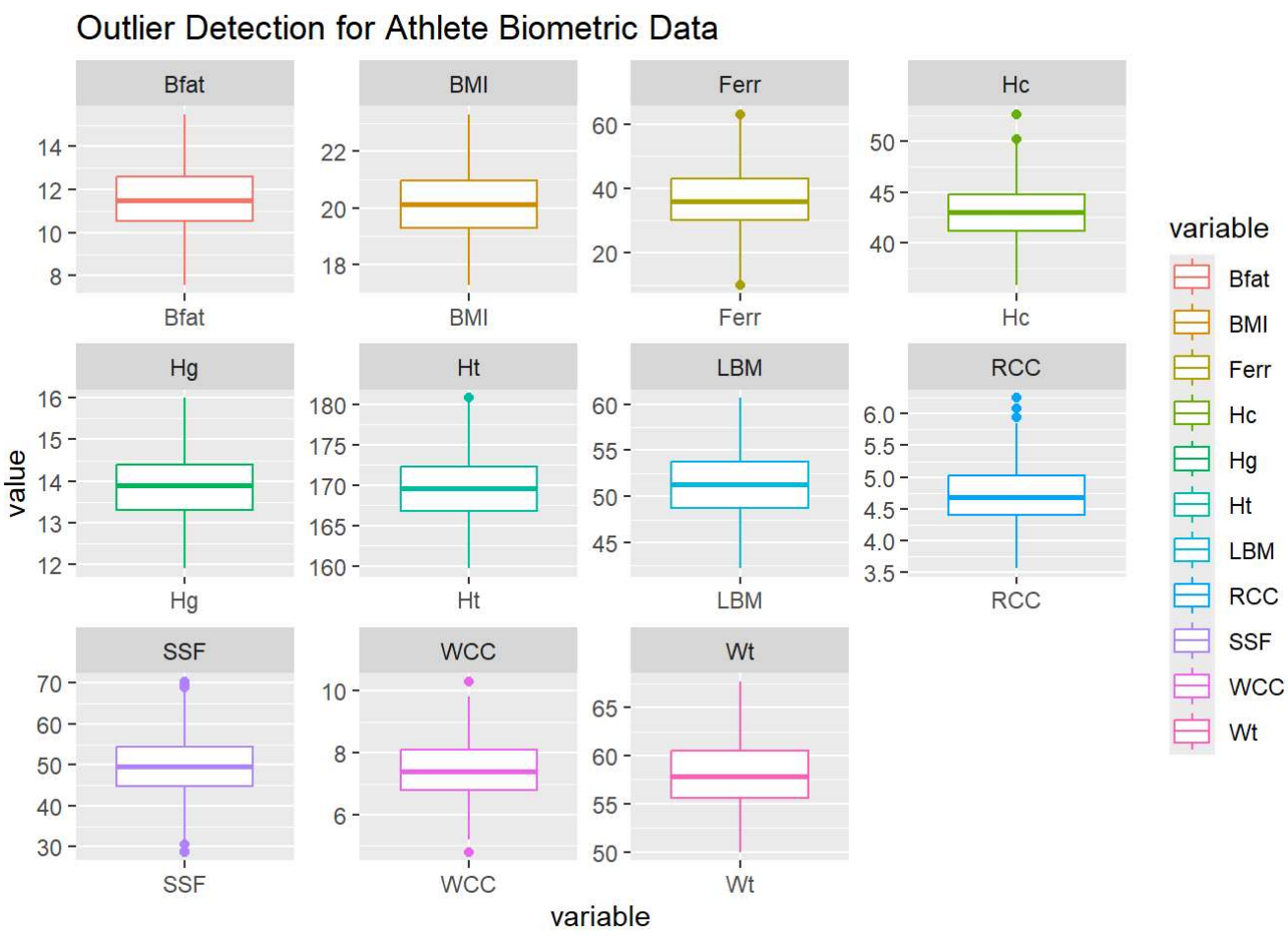
```
##        id             Ht              Wt             LBM             RCC
##  Min.   :  1   Min.   :159.7   Min.   :50.00   Min.   :42.19   Min.   :3.560
##  1st Qu.:129   1st Qu.:166.8   1st Qu.:55.60   1st Qu.:48.75   1st Qu.:4.400
##  Median :257   Median :169.6   Median :57.90   Median :51.27   Median :4.690
##  Mean   :257   Mean   :169.7   Mean   :58.01   Mean   :51.28   Mean   :4.724
##  3rd Qu.:385   3rd Qu.:172.3   3rd Qu.:60.50   3rd Qu.:53.79   3rd Qu.:5.020
##  Max.   :513   Max.   :180.8   Max.   :67.70   Max.   :60.68   Max.   :6.250
##       WCC              Hc              Hg             Ferr
##  Min.   : 4.800   Min.   :35.90   Min.   :11.9   Min.   :10.00
##  1st Qu.: 6.800   1st Qu.:41.20   1st Qu.:13.3   1st Qu.:30.00
##  Median : 7.400   Median :43.00   Median :13.9   Median :36.00
##  Mean   : 7.471   Mean   :43.06   Mean   :13.9   Mean   :36.31
##  3rd Qu.: 8.100   3rd Qu.:44.80   3rd Qu.:14.4   3rd Qu.:43.00
##  Max.   :10.300   Max.   :52.60   Max.   :16.0   Max.   :63.00
##       BMI             SSF             Bfat            Sport
##  Min.   :17.27   Min.   :28.70   Min.   : 7.57   Length   :513
##  1st Qu.:19.30   1st Qu.:44.90   1st Qu.:10.54   N.unique :  2
##  Median :20.11   Median :49.70   Median :11.52   N.blank  :  0
##  Mean   :20.16   Mean   :49.85   Mean   :11.57   Min.nchar:  4
##  3rd Qu.:20.97   3rd Qu.:54.40   3rd Qu.:12.59   Max.nchar:  6
##  Max.   :23.31   Max.   :70.40   Max.   :15.50
```

```
#Step 2: Check BMI values (Body Mass Index = weight / height^2).
athletes |>
  select(id,Wt,Ht,BMI) |>   # select the needed data.
  mutate(n_BMI=Wt/(Ht/100)^2) |> # use the formula to create a new column for checking.
  mutate(error=BMI-n_BMI) |>  # calculate the difference between the two values.
  slice_max(error,n=3)   # choose the top 3 largest differences.
```

```
## # A tibble: 3 × 6
##      id    Wt    Ht   BMI n_BMI   error
##   <dbl> <dbl> <dbl> <dbl> <dbl>   <dbl>
## 1   141  56    171.  22.3  19.2 3.09
## 2     1  55.2  168.  19.5  19.5 0.00498
## 3   290  60.3  170   20.9  20.9 0.00495
```

```
# According to the BMI formula, we can identify that the athlete with id 141 has an incorrect BMI value.
# The recorded value is 22.31, which is wrong due to a calculation error.
# The BMI does not match the corresponding weight and height values.
# The correct BMI for this athlete should be 19.22.
```

```
# Step 3: Use boxplot to find incorrect values.
athlete_long <- athletes |>
  pivot_longer(
    cols = c("Ht", "Wt", "LBM", "RCC", "WCC", "Hc", "Hg", "Ferr", "BMI", "SSF", "Bfat"),
    # Convert wide data to long format
    names_to = "variable",  # Define the column name.
    values_to = "value"     # Define the column values.
  )
  athlete_long  |>        # Use the new dataset to create the plot.
    ggplot(mapping=aes(x=variable,y=value))+       # Start plot.
    geom_boxplot(mapping=aes(col=variable)) +   # Create boxplots to identify potential outliers and double-check for errors.
    facet_wrap(vars(variable), scales="free") + # Split plots into different panels for better comparison.
    labs(title = "Outlier Detection for Athlete Biometric Data")
```




```
# Observations from the plots indicate the following:
# - Ferr includes one upper outlier and one lower outlier;
# - Hc includes two upper outliers;
# - Ht includes one upper outlier;
# - RCC includes multiple (at least three) upper outliers;
# - SSF includes two upper outliers and two lower outliers;
# - WCC includes one upper outlier and one lower outlier
# All the upper and lower outliers appear to be within a normal range.
```

2.2. Best three variables for separating sprinter and 400m groups

```
# Step 1: Compare the difference between median and mean.
long_athlete <-athletes |>
  pivot_longer(      # In order to get different sports in different blood count data.
   c("Ht", "Wt", "LBM", "RCC", "WCC", "Hc", "Hg", "Ferr", "BMI", "SSF", "Bfat"),  # Convert wide data to long format
   names_to="variable",    # Define the column name.
   values_to="value")       # Use the new dataset for analysis.
long_athlete |>
  group_by(Sport,variable) |> # Divide all sports data into groups.
  summarise(Mean=mean(value,na.rm=TRUE),
            Median=median(value,na.rm=TRUE)) |>  # Calculate mean and median values to compare differences between sports.
pivot_wider(names_from=Sport,values_from = c(Mean,Median)) |># Convert long data to wide format; sports become. column names.
mutate(d_mean=Mean_400m-Mean_Sprint,d_median=Median_400m-Median_Sprint) |> # Add two columns to clearly show the differences.
select(variable,d_mean,d_median)|>   # keep only relevant variables to compare 400m specialists and sprinters.
arrange(d_mean)  #sort rows in ascending order by value.
```

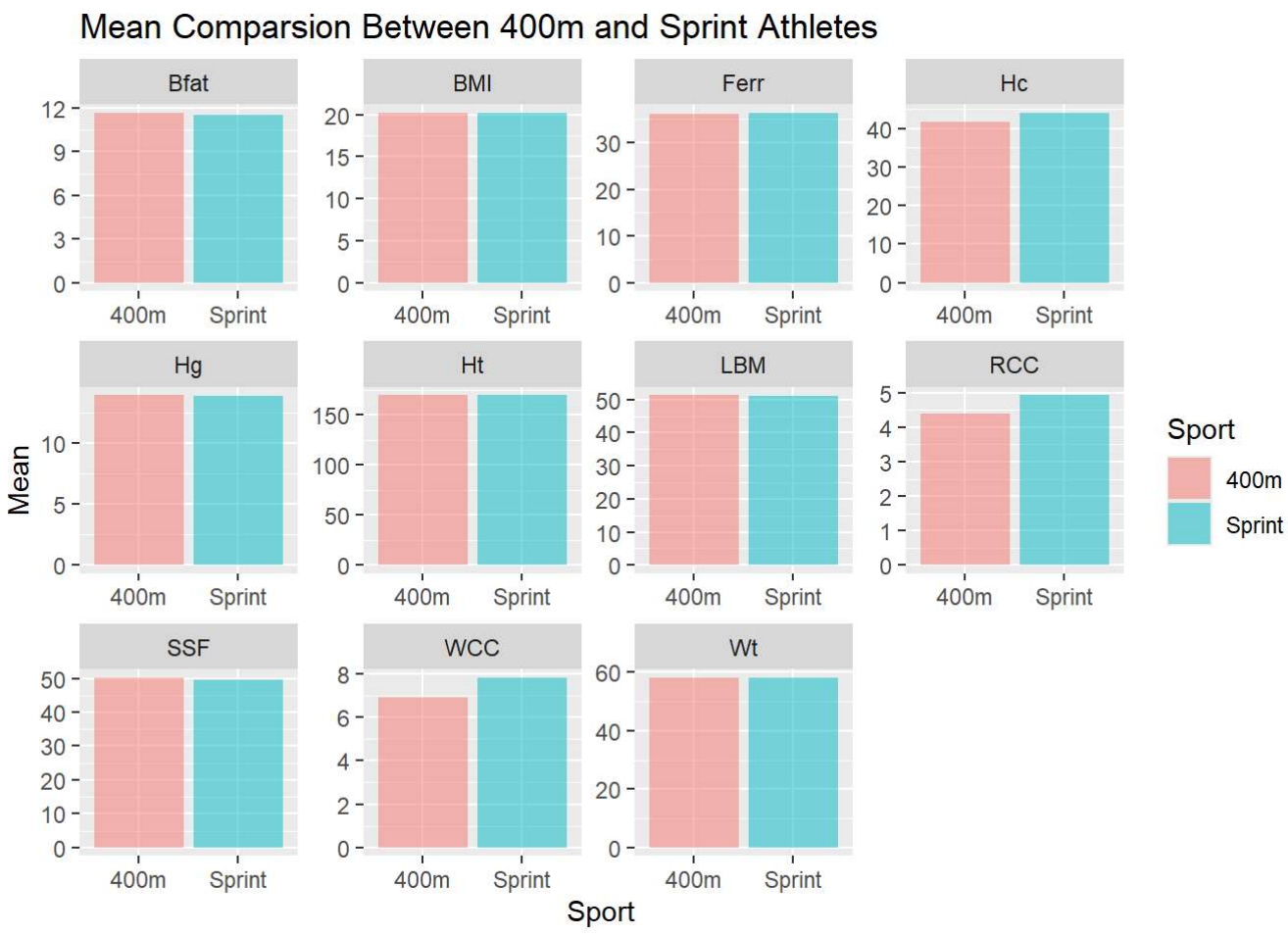
```
## `summarise()` has regrouped the output.
## ℹ Summaries were computed grouped by Sport and variable.
## ℹ Output is grouped by Sport.
## ℹ Use `summarise(.groups = "drop_last")` to silence this message.
## ℹ Use `summarise(.by = c(Sport, variable))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```

```
## # A tibble: 11 × 3
##    variable   d_mean d_median
##    <chr>       <dbl>    <dbl>
##  1 Hc       -2.47     -2.5
##  2 WCC      -0.895    -0.900
##  3 RCC      -0.534    -0.515
##  4 Ferr     -0.235    -1
##  5 Ht       -0.00749  -0.1000
##  6 Hg        0.0219    0
##  7 BMI       0.0771    0.145
##  8 LBM       0.0785   -0.480
##  9 Bfat      0.181     0.0750
## 10 Wt        0.230    -0.25
## 11 SSF       0.630    -0.1000
```

```
# Hc: the first high difference is the mean; the difference between the two groups is quite obvious.
# Sprinters have higher Hc values than 400m specialists.
# The second high difference is WCC; sprinters are higher than the 400m group.
# The third difference is SSF; for this variable, 400m specialists are higher than sprinters.
# Therefore, these three variables (Hc, WCC, and SSF) help to separate sprinters and 400m specialists.
```

```
# Step 2: Use mean bar plot to identify variables with differences.
long_athlete <-athletes |>
pivot_longer(      # In order to get different sports in different physical measurements and blood count data.
   c("Ht", "Wt", "LBM", "RCC", "WCC", "Hc", "Hg", "Ferr", "BMI", "SSF", "Bfat"),  # Convert wide data to long format
   names_to="variable",    # Define the column name.
   values_to="value")     # Use the new dataset for plotting.
long_athlete |>
   group_by(Sport,variable) |>  # Prepare data for plotting
summarise(Mean=mean(value,na.rm=TRUE) ) |> # Calculate mean to compare differences between two sports.

ggplot(mapping=aes(x=Sport,y=Mean,fill=Sport))+  # Compare two sport groups using mean values with different colors.
      geom_col(alpha=0.5)+
      facet_wrap(vars(variable),scales="free")+  # Use facet_wrap to split by 11 variables.
      labs(title = "Mean Comparsion Between 400m and Sprint Athletes") # Add plot title.
```

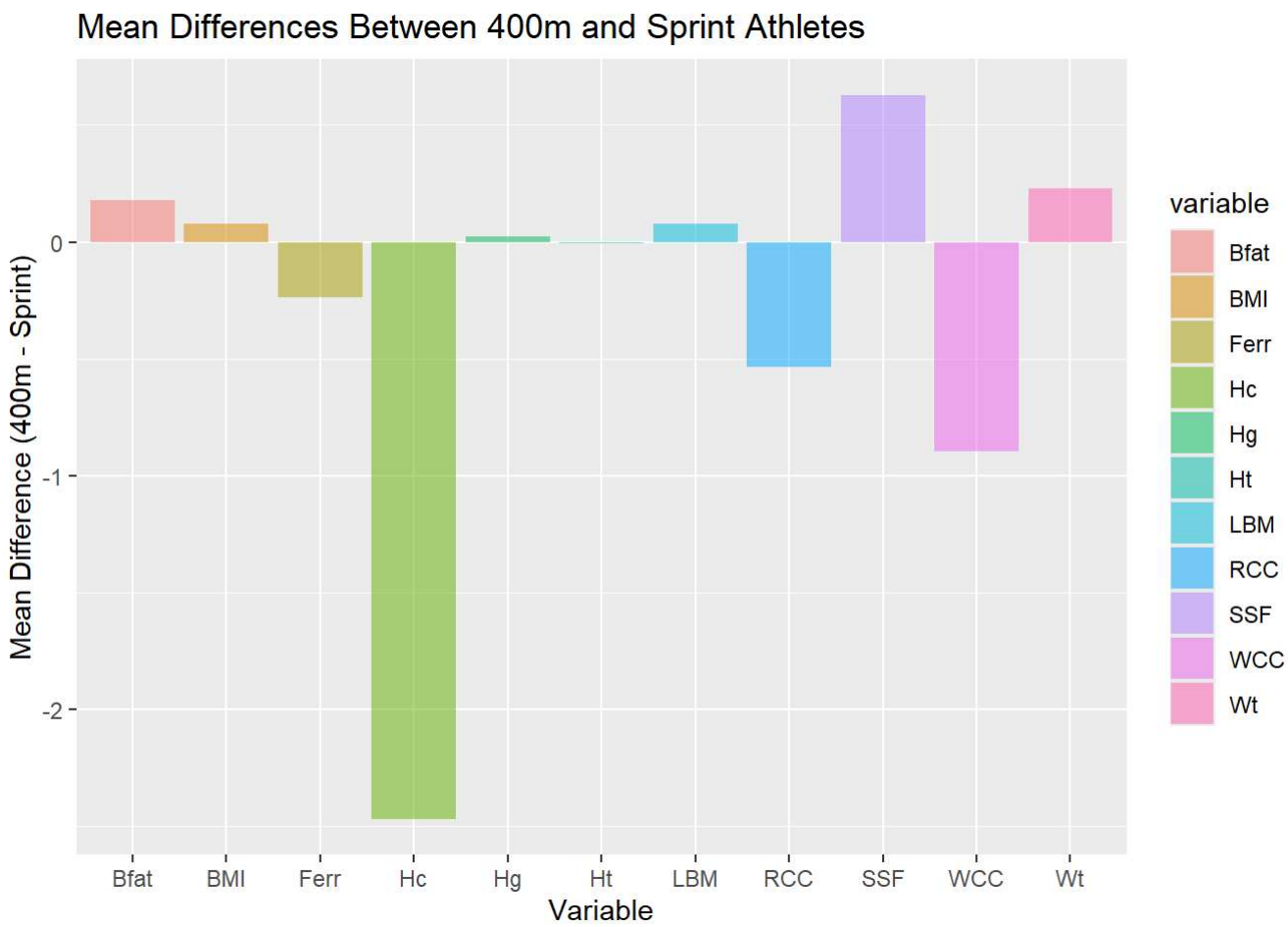
```
## `summarise()` has regrouped the output.
## ℹ Summaries were computed grouped by Sport and variable.
## ℹ Output is grouped by Sport.
## ℹ Use `summarise(.groups = "drop_last")` to silence this message.
## ℹ Use `summarise(.by = c(Sport, variable))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```



```
# By inspecting the plot directly, the largest differences between 400m and Sprint athletes are in WCC, RCC, and Hc.
# For SSF, although the numerical difference is large, the distance between sport groups is not obvious due to the scale.
# Therefore, in the next step, the mean difference will be used to make the differences more clearly visible.
```

```
# Step 3: Mean difference bar plot
long_athlete <-athletes |>
pivot_longer(      # In order to get different sports in different physical measurements and blood count data.
    c("Ht", "Wt", "LBM", "RCC", "WCC", "Hc", "Hg", "Ferr", "BMI", "SSF", "Bfat"),  # Convert wide data to long format.
    names_to="variable",     # Define the column name.
    values_to="value")       # Use the new dataset for plotting.
long_athlete |>
  group_by(Sport,variable) |>  # Prepare data for plotting.
  summarise(Mean=mean(value,na.rm=TRUE) ) |> # Calculate mean to compare differences between two sports.

# Convert to wide format so each variable has separate columns for each sport.
pivot_wider(names_from="Sport",
            values_from="Mean") |>
mutate(diff=`400m`-Sprint) |> # Calculate the difference between 400m and Sprint.
select(variable,diff) |> # Keep only relevant values.
arrange(diff) |> # Sort differences in ascending order.

# Create the bar plot of mean differences.
ggplot(mapping=aes(x=variable,
                   y=diff,
                   fill= variable))+
geom_col(alpha=0.5)+
labs(                          # Add axis labels,title with labs().
    title = "Mean Differences Between 400m and Sprint Athletes",
    x = "Variable",
    y = "Mean Difference (400m - Sprint)"
  )
```

```
## `summarise()` has regrouped the output.
## ℹ Summaries were computed grouped by Sport and variable.
## ℹ Output is grouped by Sport.
## ℹ Use `summarise(.groups = "drop_last")` to silence this message.
## ℹ Use `summarise(.by = c(Sport, variable))` for per-operation grouping
##   (`?dplyr::dplyr_by`) instead.
```




```
# From this plot, it is clear that in some aspects, Sprinters outperform 400m athletes, such as: Hc (highest difference), WCC (second largest difference).
# In SSF, 400m athletes outperform Sprinters (third largest difference).
# Therefore, the best three variables to separate Sprinters and 400m athletes are: Hc, WCC, and SSF.
```